



Programming Java

Inheritance

Incheon Paik



Contents

- Subclasses
- Inheritance & Variables
- Method Overriding
- Inheritance & Methods
- Inheritance & Constructors
- Class Modifier
- Variables Modifier
- Constructor Modifier
- Method Modifier
- The Object and Class Classes

Subclasses

Inheritance of a Class

```
class clsName2 extends clsName1 {
    // Body of class
}
```

Declaration of Variable of a Class

```
clsName varName;
```

```
class X {
}
class X1 extends X {
}
class X2 extends X {
}
class X11 extends X1 {
}
```

Instantiating

Inherit

```
class X12 extends X1 {
}
class X21 extends X2 {
}
class X22 extends X2 {
}
class InheritanceHierarchy {
    public static void main(String args[]) {
        X x;
        System.out.println("Instantiating X");
        x = new X();
        System.out.println("Instantiating X1");
        x = new X1();
        System.out.println("Instantiating X11");
        x = new X11();
        System.out.println("Instantiating X12");
        x = new X12();
        System.out.println("Instantiating X2");
        x = new X2();
        System.out.println("Instantiating X21");
        x = new X21();
        System.out.println("Instantiating X22");
        x = new X22();
    }
}
```

Subclasses

Reference to a Subclass Variable

```
super.varName
```

Inherited Variables

```
class W {
    float f;
}
class X extends W {
    StringBuffer sb;
}
class Y extends X {
    String s;
}
class Z extends Y {
    Integer i;
}
```

Inherit

```
class Wxyz {
    public static void main(String args[]) {
        Z z = new Z();
        z.f = 4.567f;
        z.sb = new StringBuffer("abcde");
        z.s = "Teach Yourself Java";
        z.i = new Integer(41);
        System.out.println("z.f = " + z.f);
        System.out.println("z.sb = " + z.sb);
        System.out.println("z.s = " + z.s);
        System.out.println("z.i = " + z.i);
    }
}
```

Result :

```
z.f = 4.567
z.sb = abcde
z.s = Teach Yourself Java
z.i = 41
```

Subclasses

```
class E {
    int x;
}

class F extends E {
    String x;
}

class Ef {
    public static void main(String args[]) {
        F f = new F();
        f.x = "This is a string";
        System.out.println("f.x = " + f.x);
        E e = new E();
        e.x = 45;
        System.out.println("e.x = " + e.x);
    }
}
```

Result :

f.x = This is a string
e.x = 45

```
class P {
    static int x;
}

class Q extends P {
    static String x;
}

class Pq {
    public static void main(String args[]) {
        P p = new P();
        p.x = 55;
        System.out.println("p.x = " + p.x);
        Q q = new Q();
        q.x = "This is a string";
        System.out.println("q.x = " + q.x);
    }
}
```

Result :

p.x = 55
q.x = This is a string

Subclasses

```
class M100 {
    int i = 100;
}

class M200 extends M100 {
    int i = 200;
    void display() {
        System.out.println("i = " + i);
        System.out.println("super.i = " + super.i);
    }
}

class SuperKeyword {
    public static void main(String args[]) {
        M200 m200 = new M200();
        m200.display();
    }
}
```

Result :

i = 200
super.i = 100



Method Overriding

Dynamic Method Dispatch Mechanism

Overloading & Overriding

Overriding : When the method with same signature is defined in Subclass

Overloading : Only the same name, but different in no. or type of parameter

```
class A1 {  
    void hello() {  
        System.out.println("Hello from A1");  
    }  
}  
  
class B1 extends A1 {  
    void hello() {  
        System.out.println("Hello from B1");  
    }  
}
```

Overridden by hello of B1

```
class C1 extends B1 {  
    void hello() {  
        System.out.println("Hello from C1");  
    }  
}  
  
class MethodOverriding1 {  
    public static void main(String args[]) {  
        C1 obj = new C1();  
        obj.hello();  
    }  
}
```

Result :
Hello from C1



Method Overriding

```
class A2 {  
    void hello() {  
        System.out.println("Hello from A2");  
    }  
}  
  
class B2 extends A2 {  
    void hello() {  
        System.out.println("Hello from B2");  
    }  
}  
  
class C2 extends B2 {  
    void hello() {  
        System.out.println("Hello from C2");  
    }  
}  
  
class MethodOverriding2 {  
    public static void main(String args[]) {  
        A2 obj = new C2();  
        obj.hello();  
    }  
}
```

Overridden by hello of B2

Result :
Hello from C2

Object of C2

Inheritance and Methods

Reference of Methods in Superclass

super.mthName(args)

```
class I1 {
    void hello(String s) {
        System.out.println("I1: " + s);
    }
}

class J1 extends I1 {
    void hello(String s) {
        super.hello(s);
        System.out.println("J1: " + s);
    }
}

class K1 extends J1 {
    void hello(String s) {
        super.hello(s);
        System.out.println("K1: " + s);
    }
}
```

```
class SuperForMethods1 {

    public static void main(String args[]) {

        System.out.println("Instantiating I1");
        I1 obj = new I1();
        obj.hello("Good morning");

        System.out.println("Instantiating J1");
        obj = new J1();
        obj.hello("Good afternoon");

        System.out.println("Instantiating K1");
        obj = new K1();
        obj.hello("Good evening");
    }
}
```

Result :

```
Instantiating I1
I1: Good morning
Instantiating J1
I1: Good afternoon
J1: Good afternoon
Instantiating K1
I1: Good evening
J1: Good evening
K1: Good evening
```

Inheritance and Constructors

Invoking of Super-Constructor

super (args)

Invoking Constructor of Same Class

this (args)

```
class S1 {
    int s1;
    S1() {
        System.out.println("S1 Constructor");
        s1 = 1;
    }
}

class T1 extends S1 {
    int t1;
    T1() {
        System.out.println("T1 Constructor");
        t1 = 2;
    }
}
```

```
class U1 extends T1 {
    int u1;
    U1() {
        System.out.println("U1 Constructor");
        u1 = 3;
    }
}

class InheritanceAndConstructors1 {
    public static void main(String args[]) {
        U1 u1 = new U1();
        System.out.println("u1.s1 = " + u1.s1);
        System.out.println("u1.t1 = " + u1.t1);
        System.out.println("u1.u1 = " + u1.u1);
    }
}
```

Result :

```
S1 Constructor
T1 Constructor
U1 Constructor
u1.s1 = 1
u1.t1 = 2
u1.u1 = 3
```

What does this mean?

Inheritance and Constructors

Invocation Order of Constructor

- 1) *Constructor of Super Class*
- 2) *Initializing Field*
- 3) *Constructor Body*

```
class S2 {
    int s2;
    S2(int s2) {
        this.s2 = s2;
    }
}

class T2 extends S2 {
    int t2;
    T2(int s2, int t2) {
        super(s2);
        this.t2 = t2;
    }
}
```

```
class U2 extends T2 {
    int u2;
    U2(int s2, int t2, int u2) {
        super(s2, t2);
        this.u2 = u2;
    }
}

class InheritanceAndConstructors2 {
    public static void main(String args[]) {
        U2 u2 = new U2(1, 2, 3);
        System.out.println("u2.s2 = " + u2.s2);
        System.out.println("u2.t2 = " + u2.t2);
        System.out.println("u2.u2 = " + u2.u2);
    }
}
```

Result :
u2.s2 = 1
u2.t2 = 2
u2.u2 = 3

Class Modifier

Qualifier of Class

abstract : Can not be instantiated
final : Cannot extend
public : Can be referenced from all classes.

Abstract Class

**A class which has abstract method(s).
It has not detailed implementation, and
can be implemented in a subclass**

```
abstract class Shape {
    void display() {
    }
}

class Circle extends Shape {
    void display() {
        System.out.println("Circle");
    }
}
```

```
class Rectangle extends Shape {
    void display() {
        System.out.println("Rectangle");
    }
}

class Triangle extends Shape {
    void display() {
        System.out.println("Triangle");
    }
}

class AbstractClassDemo {
    public static void main(String args[]) {
        Shape s = new Circle();
        s.display();
        s = new Rectangle();
        s.display();
        s = new Triangle();
        s.display();
    }
}
```

Result :

Circle
Rectangle
Triangle

Variable Modifier

Variable Modifier

final : a constant

private : can be accessed only in the same class

protected : can be accessed from subclass and the same package

public : Can be referenced from all classes.

static : not instance variable

transient : variable which is not a part of consistent state of class

volatile : protect of optimization such as constant propagation.

```
class L {
    static final int x = 5;
}

class FinalVariable {
    public static void main(String args[]) {
        System.out.println(L.x);
    }
}
```

Result :

5

Constructor Modifier

Qualifier of Constructor

private : can be accessed only in same class

protected : can be accessed from subclass and same package

public : Can be referenced from all classes.

```
class Person {
    String name;
    int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    private Person() {
    }
}
```

```
class PrivateConstructor {

    public static void main(String args[]) {

        // public Constructor Invocation
        Person p1 = new Person("John", 30);
        System.out.println(p1.name);
        System.out.println(p1.age);

        // private constructor invocation
        // Person p2 = new Person();
    }
}
```

What is the result of compilation when we removed this comment?

Current Result :

John
30

Method Modifier

Method Modifier

abstract : method without implementation

final : cannot override

native : Machine code usage such as C or C++

private : can be accessed only in the same class

protected : can be accessed from subclass and the same package

public : Can be referenced from all classes.

static : not instance method

synchronized : when executed, lock among threads

```
abstract class JetPlane {
    abstract int numEngines();
}
class DC8 extends JetPlane {
    int numEngines() {
        return 4;
    }
}
```

```
class DC10 extends JetPlane {
    int numEngines() {
        return 3;
    }
}

class JetPlanes {
    public static void main(String args[]) {
        System.out.println(new DC8().numEngines());
        System.out.println(new DC10().numEngines());
    }
}
```

Result :

4

3

Method Modifier

```
class Singleton {
    static Singleton singleton;

    private Singleton() {
    }

    public static Singleton getInstance() {
        if (singleton == null)
            singleton = new Singleton();
        return singleton;
    }
}

class SingletonDemo {
    public static void main(String args[]) {
        Singleton s1 = Singleton.getInstance();
        Singleton s2 = Singleton.getInstance();
        if (s1 == s2)
            System.out.println("Equal");
        else
            System.out.println("Not equal");
    }
}
```

Result :

Equal



The Object & Class Classes

Object Class : Most Top class in Java

equals() method

boolean equals(Object obj)

getClass() method

Class getClass()

toString() method

String toString()

Class Class

getName() method

String getName()

getSuperClass() method

Class getSuperClass()

forName() method

Static Class forName (String
clsName) throws
ClassNotFoundException



The Object & Class Classes

```
class ClassDemo {  
  
    public static void main(String args[]) {  
  
        Integer obj = new Integer(8);  
        Class cls = obj.getClass();  
        System.out.println(cls);  
    }  
}
```

Result :
Class java.lang.Integer